

BlushBite

End-to-End Technology Stack

Production-Grade Architecture — Zero Latency — All on Railway

Phases 1 → 3 • Frontend to Database • Auth to AI • Media to Real-Time

Version 2.0 — Updated for Railway-First Infrastructure

CONFIDENTIAL — INTERNAL TECHNICAL REFERENCE

1. Architecture Philosophy

BlushBite is a media-heavy, real-time, AI-powered adult platform. Every technology decision is driven by four hard requirements: near-zero perceptible latency on every user interaction, a buttery-smooth feed experience on both mobile and desktop, the ability to scale from 0 to 100,000+ concurrent users without rearchitecting, and minimal DevOps burden on a lean team. Railway provides the unified infrastructure platform that makes all of this possible from a single control plane.

Railway-First Architecture All backend services — frontend, API, database, cache, worker, and socket server — run inside a single Railway project. Services communicate over Railway’s private network, which means internal calls between the API and the database have effectively zero network overhead. No VPC setup, no Kubernetes YAML, no load balancer configuration.

Design Principle	How It Is Applied in BlushBite
Edge-First Rendering	Next.js App Router with React Server Components. HTML is rendered on the server and streamed to the browser. The user sees above-fold content before JavaScript has even parsed. TTFB target: < 100ms.
Zero Waterfall Fetching	All critical data for a page is fetched in parallel on the server using Promise.all inside Server Components. No sequential loading spinners on first render.
Optimistic UI	Every user action — like, save, comment, scroll next — updates the UI instantly before the server responds. Server confirmation happens silently in the background. Zero perceived latency.
CDN-First Media	Every escort photo, 15-second video clip, and audio file is served from Cloudflare R2 via CDN edge cache. Media requests never reach the Railway servers. Served from the edge node closest to the user, typically <20ms.
Private Network Queries	Railway’s internal network connects the API directly to PostgreSQL and Redis over a private hostname. No TLS overhead, no public internet routing. DB query RTT: ~1–2ms.
Async Everything Heavy	AI voice generation, ML embedding computation, email sending, feed recalculation, and content moderation all happen in background BullMQ jobs. The API response never waits for heavy work.
Stateless API	The Hono.js API is fully stateless. All session state lives in signed JWTs. Any number of Railway API replicas can serve any request. Horizontal scaling with zero coordination overhead.

2. Complete Stack — Layer by Layer

Every technology used in the BlushBite platform across all three phases, with the rationale for each choice and operational notes for the team.

Layer	Technology	Version	Purpose & Rationale	Team Notes
FRONTEND — What the User Sees				
Framework	Next.js App Router	v14 / v15	Full-stack React with RSC, SSR, Streaming, and file-based routing. Renders HTML at the server edge. Fastest possible TTFB. Native support for image optimisation, fonts, and metadata.	Default to Server Components everywhere. 'use client' only when hooks or browser APIs are needed. Never use pages/ router.
UI Runtime	React	v18+	Component model. Server Components for all data-fetching views. Client Components only for interactive elements like the audio player, like button, save toggle.	Keep Client Component trees shallow. Focus on the interactive leaf, not the whole page.
Styling	Tailwind CSS	v3	Utility-first CSS. Zero runtime. All dark-luxury design tokens (navy palette, rose-gold accents, spacing, type scale) defined in tailwind.config.ts. No CSS-in-JS runtime overhead.	Never write custom CSS files. All styles use className. Use dark: prefix for dark mode variants.
Component Library	shadcn/ui + Radix UI	Latest	Accessible, headless, fully customisable components. Dialogs, dropdowns, sheets, sliders all use Radix primitives under the hood. Zero default shadcn styling shipped to users.	Run 'npx shadcn-ui add [component]'. Customise in components/ui/ only.
Animations	Framer Motion	v11+	GPU-accelerated animations for feed card entrances, page transitions, audio player slide-up, Bridge row reveal, and like/save micro-interactions. Spring physics for premium feel.	Use motion.div with variants for page transitions. AnimatePresence for mount/unmount. Animate layout properties — use transition
Feed Virtualisation	TanStack Virtual	v3	Only renders DOM nodes that are currently in the viewport. Essential for the home feed which can have hundreds of cards. Zero jank on scroll regardless of feed length.	Use useVirtualizer hook on the feed container. Pair with Intersection Observer for lazy load.
Data Fetching (client)	TanStack Query (React Query)	v5	Handles all client-side data fetching, caching, background refetch, pagination, and optimistic updates. Bootstrapped with server-prefetched data so no loading spinners on first render.	useQuery for reads, useMutation + useOptimistic for optimistic updates. Set staleTime per endpoint on data freshness requirements.

Layer	Technology	Version	Purpose & Rationale	Team Notes
Global State	Zustand	v4+	Lightweight global state store. Manages: auth session, user preferences, mood slider state, audio player state (current track, progress, playing), and notification count.	One store per domain (authStore, pl prefsStore). No redux boilerplate. Pe playerStore to localStorage so playb navigation.
Forms	React Hook Form + Zod	Latest	Zero re-render forms. Zod schemas defined in packages/shared and used for both frontend validation and backend API validation. Used for: booking form, onboarding wizard, story post form, companion upload form.	Never use useState for form fields. S schemas between frontend and back packages/shared.
Infinite Scroll	Intersection Observer API	Native	Companion directory and story feeds use IntersectionObserver to trigger next-page fetch 200px before the user reaches the bottom. No pagination buttons.	Attach observer to a sentinel div at th the list. Disconnect on unmount.
Video Autoplay	IntersectionObserver + HTMLVideoElement	Native	Escort 15-second video clips auto-play muted when 80% in viewport, pause when scrolled past. Sound plays on tap.	Use data-src for lazy video src. Set s entering viewport threshold.
Icons	Lucide React	Latest	Tree-shakeable. Only used icons are bundled. Consistent with the dark-luxury design language.	Import individually: import { Heart } fr react'. Never import the full set.
Fonts	next/font (Google Fonts)	Built-in	Self-hosted. Zero layout shift. Zero external font requests. Primary: Inter (UI). Display: Playfair Display (headings, companion names).	Defined in app/layout.tsx. CSS variat globally. Never load fonts via <link>.
BACKEND — The Server Layer				
Runtime	Node.js	v20 LTS	JavaScript runtime for all backend services. Single language across full stack. Massive ecosystem. Handles async I/O extremely efficiently for API + real-time patterns.	Use v20 LTS. Enable --experimental for Jest/Vitest ESM support.
API Framework	Hono.js	v4+	Ultra-lightweight HTTP framework. 3× faster than Express in benchmarks. Edge-compatible (can run on Cloudflare Workers if needed in future). Minimal middleware overhead.	Use Hono for all REST routes and w endpoints. Mount tRPC adapter insio
Type-Safe API	tRPC	v11	End-to-end TypeScript type safety between Next.js frontend and Node backend. No API contract files to maintain. Frontend gets full autocomplete on every API call.	All procedures in server/routers/. Use protectedProcedure for auth-gated c Input/output validated with Zod scher packages/shared.

Layer	Technology	Version	Purpose & Rationale	Team Notes
ORM	Drizzle ORM	Latest	Type-safe SQL ORM. Generates actual SQL queries (inspectable). Faster than Prisma for complex joins. Schema-first: define tables in packages/db/schema.ts, generate migrations.	Never write raw SQL in app code. Use query builder. Run migrations with 'drizzle-kit migrate' in C
Input Validation	Zod	v3	Runtime schema validation on all API inputs. Schemas live in packages/shared and are imported by both frontend forms and backend procedures. Single source of truth for data shapes.	Define all schemas in packages/shared. Export types with z.infer<typeof sche
Background Jobs	BullMQ	v5+	Redis-backed job queue. All heavy async work: AI audio generation, OpenAI embedding calls, email sending, feed weight recalculation, ML vector updates, content moderation pre-screening.	Separate Railway service (apps/work pull jobs from Redis queues. Define j packages/shared/jobs.ts.
File Uploads	Signed Cloudflare R2 URLs	Custom	Client requests a pre-signed R2 upload URL from the API. Client uploads directly to R2. API stores the resulting CDN URL in the database. Backend never handles binary file data.	Generate signed URL in tRPC upload procedure. Set Content-Type and ma the signature. Companion video: ma enforced client-side before upload.
Text Processing (Hinglish)	Custom pre-processor + ElevenLabs phoneme hints	Custom	Before Hinglish text is sent to ElevenLabs, a pre-processing step normalises Roman Hindi transliterations, injects ElevenLabs SSML phoneme tags for common Hindi words, and adds natural pause markers.	Maintain a Hindi word → IPA phoner packages/shared/hinglish.ts. Expand words appear in content.
INFRASTRUCTURE — All on Railway (Single Project)				
Next.js Frontend	Railway Service: blushbite-web	Phase 1	Hosts the Next.js app. Auto-deploy from GitHub main branch. EU Frankfurt region. Railway detects Next.js automatically and configures the build. PORT env var set by Railway.	Set RAILWAY_HEALTHCHECK_TIMEOUT for Next.js cold starts. Use nixpacks
Hono API Server	Railway Service: blushbite-api	Phase 1	Node.js + Hono + tRPC. Connects to Postgres and Redis via Railway private network hostnames {{{Postgres.DATABASE_URL}}}, {{{Redis.REDIS_URL}}}. No public DB exposure.	Enable Railway's HTTP health check Set NODE_ENV=production. Private means DB calls never leave the Rail center.
PostgreSQL	Railway Managed Postgres	Phase 1	Managed PostgreSQL 16. Automated daily backups. pgvector extension enabled on first deploy. Private network URL	Run 'CREATE EXTENSION IF NOT vector' on first deploy. Set connection 20 in Drizzle config. Railway provides string as DATABASE_URL.

Layer	Technology	Version	Purpose & Rationale	Team Notes
Redis	Railway Managed Redis	Phase 1	used by API and Worker. No public internet access to DB. Managed Redis 7. Used for: JWT session cache, BullMQ job queues, rate limiting counters, feed cache, real-time presence keys (Phase 3). Private network URL.	Set maxmemory-policy to allkeys-lru. Separate Redis DB index from session data. Railway provides REDIS_URL.
BullMQ Worker	Railway Service: blushbite-worker	Phase 1	Separate Railway service. Consumes jobs from Redis queues. Handles: AI audio jobs (ElevenLabs), embedding jobs (OpenAI), email jobs (Resend), feed recalculation, moderation pre-screen. Scales independently of API.	Worker can be scaled to multiple Railway instances if job queue depth grows. Each worker is stateless. Uses same DATABASE_URL and REDIS_URL as API.
Socket.io Server	Railway Service: blushbite-socket	Phase 3	Dedicated WebSocket service. Sticky sessions enabled in Railway settings (required for Socket.io). Redis adapter configured for multi-replica fanout via Pub/Sub.	Add Railway service in Phase 3 only. 'Sticky Sessions' in Railway service settings. @socket.io/redis-adapter with the existing instance.
Coturn TURN Server	Hetzner VPS CX21 (Netherlands)	Phase 3	The only infrastructure outside Railway. Required for WebRTC audio/video calls when peer-to-peer connection is blocked by NAT or corporate firewalls. 20TB/month bandwidth included on Hetzner CX21.	Install coturn via apt. Configure with TURN credentials. Expose UDP 3478 and TCP 443. Use TLS. Encrypt SSL. Upgrade to CX31 when call volume exceeds 200.
MEDIA & CDN — Cloudflare				
Object Storage	Cloudflare R2	Pay-as-you-go	Stores: companion profile photos (up to 20 per companion), escort 15-second video clips (up to 5 per companion), escort audio stories, user audio uploads, AI-generated audio files (cached after first generation), UGC profile pictures. Zero egress fees — no cost to serve files.	Bucket structure: /companions/[id]/photos/, /companions/[id]/videos/, /companions/[id]/audio-generated/, /users/[id]/. Pre-signed URLs expire in 15 minutes.
CDN & DDoS	Cloudflare CDN + Pro	Pro plan	All R2 assets served through a Cloudflare CDN subdomain (media.blushbite.com). Edge caches all media at 300+ global PoPs. Auto-converts images to WebP/AVIF based on browser Accept header. 40–60% smaller than JPEG.	Set Cache-Control: public, max-age=31536000 for all R2 objects. Cloudflare caches for WAF, DDoS mitigation, and bot management. Always active on Pro plan.
Video Processing	Cloudflare Stream	Usage-based	Handles escort 15-second video clips. On upload: transcodes to multiple bitrates (adaptive HLS), generates auto-thumbnail, strips metadata. Muted autoplay in	Use Stream upload API for companion videos. Ingest. Store Stream video ID in DB. Render via HLS player component. Stream handles all encoding.

Layer	Technology	Version	Purpose & Rationale	Team Notes
			feed via HLS.js. Full sound on profile page.	
Image Optimisation	Next.js Image component	Built-in	All companion photos and profile pictures rendered via next/image. Automatic: WebP/AVIF conversion, lazy loading, blur placeholder (eliminates CLS), responsive srcSet generation.	Always provide width and height prop, priority prop on hero companion card, placeholder='blur' with blurDataURL for LCP images.
AUTH & IDENTITY				
Auth Framework	NextAuth.js (Auth.js v5)	v5	Handles Google OAuth 2.0 flow, JWT session creation, CSRF protection, and secure cookie management. Native integration with Next.js App Router and Server Components.	Use getSession() in Server Components and API route handlers. Never decode manually. Use NextAuth callbacks to claims (age_verified, alias, role).
OAuth Provider	Google OAuth 2.0	Free	Single sign-in method. Only email and name captured. Anonymous alias generated server-side on first sign-in. Real email stored as bcrypt hash only — never displayable.	Configure OAuth consent screen in Google Console. Set authorised redirect URL to production URL + localhost. Only request profile scopes.
Sessions	JWT + httpOnly Secure Cookies	Via NextAuth	Stateless sessions. Short-lived access token (15 min) + refresh token rotation. Both stored as httpOnly, Secure, SameSite=Strict cookies. Cannot be read by JavaScript. Cannot be stolen by XSS.	Set NEXTAUTH_SECRET as a 64-character random hex string in Railway env. Rotate every 30 days. Enable useSecureCookies=true in config.
Age Gate	Next.js Middleware	Custom	Server-side. Runs on every request to protected routes before the page renders. Checks age_verified claim in JWT. Redirects to age confirmation page if not set. Cannot be bypassed client-side.	Defined in middleware.ts at project root. Uses NextAuth getToken() to read JWT claims. Route matcher: all routes except /auth and /api/auth.
Anonymous Alias	UUID-based generator	Custom	On first sign-in, server generates an alias: @[adjective]-[noun] from a curated wordlist (e.g. @velvet-wanderer, @midnight-echo). Stored in users table. Shown instead of real name everywhere on platform.	Wordlist in packages/shared/aliases. Generates unique combinations. User can request new generation per 30 days (rate-limited by Redis).
Rate Limiting	Upstash Redis Rate Limit (via Railway Redis)	Custom	Per-IP and per-user rate limiting on all API endpoints using a sliding window counter in Railway's Redis. Prevents brute-force, scraping, and booking spam.	Implement in Hono middleware. Key: ip userId:endpoint. Limits: 100 req/min on public routes, 30 req/min on auth-gateways.
AI & MACHINE LEARNING				

Layer	Technology	Version	Purpose & Rationale	Team Notes
AI Voice Engine	ElevenLabs API	Creator plan	Converts written stories and confessions to immersive AI audio. Three voice styles configurable per story: Husky Femme, Calm Masculine, Gender-Neutral. Pacing and pitch adjustable per mood tag category (e.g. Soft Intimacy = slower, Power Exchange = faster).	API call made via BullMQ job (never resulting MP3 in Cloudflare R2 immediate generation. Subsequent listens served from cache — zero repeat API cost.
Hinglish AI Audio	ElevenLabs + custom pre-processor	Custom	Pre-processing pipeline normalises Roman Hindi transliterations before sending to ElevenLabs. Injects SSML <phoneme> tags for common Hindi words (e.g. 'yaar', 'dil', 'ishq'). Pronunciation dictionary maintained in packages/shared/hinglish-phonemes.ts.	Test each new Hindi word addition against ElevenLabs output before adding to pronunciation dictionary. A/B test Hinglish audio quality with control sample group before full rollout in Production.
Text Embeddings	OpenAI text-embedding-3-small	API	Generates 1536-dimension semantic vectors for: user preference profiles (built from onboarding tags + engagement history), companion profiles (tags + bio + story content), and every published story/confession. Stored in PostgreSQL with pgvector extension.	Called via BullMQ embedding job. Not called OpenAI synchronously in an API handler on content creation/update. Re-embed companion profiles when they add new content.
Vector Database	pgvector (Postgres extension)	Built into Railway Postgres	Enables cosine similarity search directly inside PostgreSQL. HNSW index on user_vectors and companion_vectors tables for <50ms similarity queries at scale. No separate vector DB service needed.	Run CREATE INDEX ... USING hnsw migration. Use drizzle's sql operator for cosine similarity: ORDER BY embedding <=> \$userVector LIMIT 20.
Recommendation Engine	Custom (pgvector + BullMQ)	Phase 2	Combines: (a) cosine similarity between user taste vector and companion/content vectors, (b) collaborative filtering signal from user_engagement table, (c) Bridge tag overlap score, (d) content freshness decay. Feed weights recalculated hourly per user via BullMQ cron job.	Store computed feed_weights per user in Redis cache (TTL: 2 hours). API reads from cache and re-computes on every request. Cache miss triggers synchronous fallback to tag-matching (Phase 1 method).
Content Moderation AI	OpenAI Moderation API	Free tier	Pre-screens all user-generated text (story submissions, comments, bio fields) before they enter the human moderation queue. Flags categories: sexual/minors, hate, violence, self-harm. Auto-rejects clearly violating content.	Called synchronously before inserting into the moderation queue (it's fast and cheap). Use it to replace human review — used to reduce human review volume.

Layer	Technology	Version	Purpose & Rationale	Team Notes
Behavioural Analytics	PostHog	Self-hosted on Railway or PostHog Cloud	Captures all user behaviour signals: scroll depth (Intersection Observer), read/listen completion rates (%), via progress events), hover events on companion cards, save/like events, session depth, Bridge click-through rates, booking funnel steps.	Self-host PostHog on Railway in Phase 3. User behaviour data within the Railway self-hosting overhead is too high, use PostHog Cloud EU region. Never send real user data to PostHog — use anonymous alias.
REAL-TIME SYSTEMS (Phase 3)				
WebSocket Server	Socket.io	v4+	Powers all real-time 1-to-1 chat between users and companions. Rooms scoped per conversation ID. Seen and typing indicators. JWT authentication on connection. Redis adapter for cross-replica fanout.	Railway service: blushbite-socket. Enforces sessions in Railway. Use io.adapter(createAdapter(redisClient)). Reuses existing Railway Redis instance.
Real-Time Presence	Redis Pub/Sub + expiring keys	Via Railway Redis	Companion online/available status stored as a Redis key with 30-second TTL. Companion client sends a heartbeat every 20 seconds to refresh TTL. Key expires automatically on disconnect — no cleanup code needed.	Key pattern: presence:companion:{id}. Value: {status, lastSeen}. Subscribe to keys to receive notifications for expiry events to push status to open chat windows.
WebRTC Signalling	Pion (Go) or mediasoup (Node)	Phase 3	Signalling server brokers the ICE candidate exchange and SDP offer/answer negotiation between user and companion browsers. Once peers connect, media flows peer-to-peer — the signalling server is no longer in the media path.	Signalling server can run as a lightweight service (mediasoup) or a small Go binary. Choose based on team's Go vs Node.js expertise level.
TURN Server	Coturn on Hetzner CX21 NL	Phase 3	Relays WebRTC media when direct peer-to-peer connection is blocked by NAT or firewall. Required for ~30–40% of real-world WebRTC connections. Hetzner NL provides 20TB/month free bandwidth.	Configure Coturn with time-limited HmacSha1 credentials (not static password). Generate credentials in the API per call session. Rotate TURN secret monthly.
Push Notifications	Web Push API + Service Worker	Phase 3	Browser push for: session reminders (10 min before scheduled call), new message alerts, companion availability notifications. Works on mobile browser without an app. No third-party push service.	Register service worker in apps/web. Store push subscriptions in DB (push_subscriptions table). Send via BullMQ job (npm package in BullMQ notification job).
PAYMENTS				
Primary Processor	Stripe	Adult MCC code	Handles: booking payments, companion listing fees, featured placement fees, premium membership subscriptions, per-	Must register under correct adult MCC code (e.g. 7999 depending on category) from Stripe Connect for companion payouts. Events processed via BullMQ job (ne-

Layer	Technology	Version	Purpose & Rationale	Team Notes
			minute audio/video call billing (Stripe Meter API in Phase 3).	
Metered Billing	Stripe Meter API	Phase 3	Tracks audio and video call minutes per user. Charges applied automatically at end of session. Stripe handles proration and billing cycle management.	Use Stripe Meter for usage-based billing. Track usage events in Stripe from the call table after session ends.
Backup Processor	CCBill or Segpay	Phase 2 prep	Adult-specialist payment processors as hot standby if Stripe restricts the account. Pre-integrate in Phase 2 so switching is fast if needed.	Keep CCBill integration behind a feature flag with 5% of traffic before full switchover.
EMAIL & NOTIFICATIONS				
Email Service	Resend	Free → Pro	Transactional emails: booking confirmations, booking request notifications to admin, moderation alerts, session reminders, welcome email on sign-up.	All email sending via BullMQ email job processor. Built with React Email in packages/email. Send email synchronously in an API endpoint.
Email Templates	React Email	Latest	Build HTML emails using React components. Consistent with platform UI. Renders to HTML+plaintext for email clients. Templates in packages/email/.	Test templates with Email Previews (in dev server). Always include plaintext fallback.
MONITORING, OBSERVABILITY & SECURITY				
Error Tracking	Sentry	Free → Team	Real-time error capture on both Next.js frontend and Node API. Source maps uploaded on deploy so stack traces show original TypeScript source. Alerts to Slack/email on new errors.	Wrap Railway deploy command with Sentry CLI. Set tracesSampleRate in production (10% of transactions sampled for performance).
Structured Logging	Axiom or Logtail	Free tier	All API requests, BullMQ job events, WebSocket events, and auth events logged as structured JSON. Searchable in real-time. Essential for debugging production issues.	Use pino logger in the API. Ship logs to Axiom or Logtail via pino transport. Log level: info in production, debug in staging.
Uptime Monitoring	BetterStack Uptime	Free	Monitors all Railway service health endpoints every 30 seconds. Alerts via SMS + email on downtime. Public status page for client transparency.	Monitor: /health on API, / on frontend. Set 15-second timeout. Ping, Postgres SELECT 1. Set 15-second timeout.
Web Application Firewall	Cloudflare WAF (Pro)	Pro plan	Blocks SQL injection, XSS, and known attack patterns at the Cloudflare edge before requests reach Railway. Custom rules to block known adult platform scrapers.	Enable OWASP Core Rule Set. Add custom rules to rate-limit booking endpoints. Geo-block jurisdictions where platform is legally restricted.

Layer	Technology	Version	Purpose & Rationale	Team Notes
Secret Management	Railway Environment Variables	Built-in	All API keys, DB credentials, OAuth secrets, and signing keys stored as Railway environment variables. Injected at runtime. Never committed to code. Separate variable sets for production and staging.	Use Railway's 'Shared Variables' for secrets across multiple services (e.g. DATABASE_URL, REDIS_URL, NEXTAUTH_SECRET). Service-specific vars set on each service individually.
Data Encryption	AES-256 at rest (Railway Postgres), TLS 1.3 in transit	Built-in	All data at rest encrypted by Railway's managed Postgres. All network traffic encrypted TLS 1.3. User preference data (sexual preferences) stored encrypted at column level using pgcrypto.	Enable pgcrypto extension in Postgres. Add preferences_json, email_hash columns in Railway env (COLUMN_ENCRYPTION_KEY). Keys rotated annually.
HTTPS Enforcement	Cloudflare Universal SSL + HSTS	Free	Automatic HTTPS for all domains. HSTS header enforced (max-age=31536000, includeSubDomains). TLS 1.3 only. HTTP requests redirected to HTTPS at Cloudflare edge.	Set HSTS in Cloudflare SSL/TLS → Settings. Enable 'Always Use HTTPS'. Minimum TLS version to 1.2 (TLS 1.3 preferred).

3. Railway Project Architecture

All services live in a single Railway project. Private networking means service-to-service calls use internal hostnames with zero internet routing overhead. The typical API → Database round-trip is 1–2ms.

Private Networking URLs Within a Railway project, services reference each other using the pattern: `${ServiceName.ENV_VAR}`. For example, the API accesses Postgres as `Postgres.DATABASE_URL` and Redis as `Redis.REDIS_URL`. These resolve to private IPs that never leave the Railway data center.

Railway Service	Phase	Internal Access	What It Does
blushbite-web (Next.js)	Phase 1	Public + private	Frontend. Auto-deploys from GitHub. Calls API via private network. Public HTTPS URL served through Cloudflare.
blushbite-api (Hono + tRPC)	Phase 1	Private + public webhooks	API server. Private network access to Postgres + Redis. Public only for Stripe webhooks and health check.
Postgres (Railway Managed)	Phase 1	Private only	Primary data store. pgvector enabled. Never exposed to public internet. Only accessible to blushbite-api and blushbite-worker.
Redis (Railway Managed)	Phase 1	Private only	Sessions, cache, BullMQ queues, rate limiting. Accessible to API, Worker, and Socket services.
blushbite-worker (BullMQ)	Phase 1	Private only	Async job processor. No public port. Connects to Postgres + Redis. Calls external APIs (ElevenLabs, OpenAI, Resend).
blushbite-socket (Socket.io)	Phase 3	Public (WebSocket)	Real-time chat. Sticky sessions enabled. Redis adapter for multi-replica. Authenticated via JWT on connection.
Hetzner CX21 VPS	Phase 3	Public (UDP/TCP)	Coturn TURN server. Only non-Railway infrastructure. Ports: UDP 3478, TCP 443.

3.1 Request Lifecycle — Home Feed Page Load

#	Where	What Happens
1	Browser → Cloudflare	DNS resolves blushbite.com → Cloudflare. Static assets (JS bundles, CSS, fonts) served from Cloudflare edge cache. Already cached from last deploy. Delivery time: <20ms globally.
2	Cloudflare → Railway (Next.js)	HTML request forwarded to Railway Next.js service. React Server Component rendering begins. Target TTFB from Railway: <80ms.
3	Next.js → Railway Postgres (private)	Server Component fetches user preferences + companion feed rows + story rows in parallel (Promise.all). Private network: ~1–2ms per query.
4	Next.js → Railway Redis (private)	Feed weight cache checked first. If cache hit (TTL 2hrs): companion ranking served from Redis instantly. Cache miss: falls back to Postgres similarity query.
5	Companion card image URLs	All companion photo URLs point to media.blushbite.com (Cloudflare R2 CDN). No images are fetched through Railway. CDN serves from nearest edge node.

#	Where	What Happens
6	RSC streaming to browser	Rendered HTML streamed to browser via React Suspense streaming. Above-fold content (hero card + first row) arrives first. Below-fold rows stream in as they resolve.
7	Client hydration	React hydrates interactive elements. TanStack Query bootstrapped with server-prefetched data. No loading spinners. Framer Motion animates card entrances.
8	Background: feed prefetch	TanStack Query prefetches page 2 of feed data 500ms after mount. User never sees a loading state when they scroll down.

Target metrics: *TTFB* < 100ms | *LCP* < 1.2s | *INP* < 100ms | *CLS* = 0.00 | *TTI* < 2s

3.2 Request Lifecycle — AI Audio Generation

#	Where	What Happens
1	User taps Listen	Client calls tRPC generateAudio mutation. API responds immediately: 'Generating...' status returned. BullMQ audio job enqueued.
2	Worker picks up job	blushbite-worker dequeues the audio job from Redis. Checks R2 cache: has this exact story+voice been generated before? If yes, returns cached URL immediately.
3	Hinglish pre-processing	If story contains Hinglish text: pre-processor normalises and injects SSML phoneme tags. Clean text sent to ElevenLabs.
4	ElevenLabs API call	Worker calls ElevenLabs text-to-speech endpoint with voice ID, stability, and similarity_boost settings tuned for the story's mood tag.
5	MP3 cached to R2	Generated audio file streamed directly to Cloudflare R2. URL stored in stories table (audio_url column). Job marked complete.
6	Client polls / SSE	Client polls tRPC audioStatus query every 2 seconds (or receives SSE event when job completes). Audio URL returned. Sticky player activates.
7	All future listens	CDN serves audio from R2 edge cache. ElevenLabs API never called again for same content. Zero repeat cost.

4. Rendering Strategy per Page

Choosing the correct rendering mode per route is critical for achieving the performance targets. The wrong choice on the home feed (client-side instead of RSC) would add 2–3 seconds to first load.

Route	Rendering Mode	Cache Behaviour	Reason
/ (Home Feed)	RSC + Streaming	Per-user, no CDN cache	Fully personalised. Cannot be cached globally. Server streams rows as they resolve. First row arrives in <100ms.
/companions	ISR	CDN 60s revalidate	Companion directory mostly static. ISR gives CDN-level performance with near-fresh data.
/companions/[id]	ISR + Partial Prerender	CDN 5min revalidate	Static shell (photos, bio, tags) prerendered. Dynamic sections (availability, session pricing) streamed.
/stories	ISR	CDN 2min revalidate	Story list changes slowly. CDN-level performance. Trending row updated at revalidation.
/stories/[id]	SSR + client cache	TanStack Query 5min stale	Story content + dynamic Bridge row at bottom. SSR for SEO. Client cache avoids re-fetch on back navigation.
/explore	Client-side + API	TanStack Query, real-time	Fully user-driven query with live filters. Must be entirely dynamic.
/me	Client-side only	No cache	Fully authenticated personal data. Saved items, preferences, bookings.
/admin/*	SSR (auth-gated)	No cache	Admin always sees live data. Route protected by middleware role check.
/chat/* (Ph 3)	Client-side + WebSocket	No cache	Real-time. Initial history loaded via tRPC, live updates via Socket.io.

4.1 Core Web Vitals Targets & How We Hit Them

Metric	Target	Technique	Measurement
LCP	< 1.2s	Hero companion card image: next/image with priority prop + rel=preload link in document head. Served from Cloudflare CDN edge.	Vercel Analytics or Railway-served Lighthouse CI
INP	< 100ms	Minimal JS on initial load. Event handlers are lightweight. Heavy computation deferred to web workers or server.	Chrome UX Report + Sentry performance
CLS	0.00	All images have explicit width/height. next/font self-hosted (zero FOUT). Skeleton loaders match exact dimensions of real content.	Lighthouse CI in GitHub Actions

Metric	Target	Technique	Measurement
TTFB	< 100ms	Next.js RSC on Railway EU Frankfurt. Redis cache hit for feed weights. DB queries run over private network (~1ms).	BetterStack Real User Monitoring
TTI	< 2.0s	Critical JS only. Dynamic imports for non-critical components (Framer Motion, audio player). Route-level code splitting automatic in Next.js.	Lighthouse CI

5. Database Schema — Core Tables

All data lives in Railway-hosted PostgreSQL. The schema is designed around the actual query patterns of the platform: personalised feed generation is the critical path, followed by companion profile reads and booking creation.

Table	Key Columns	Purpose & Design Notes
users	id, alias, email_hash, preferences_json_encrypted, age_verified, created_at	Anonymous by design. Real email stored as one-way hash only. preferences_json encrypted with pgcrypto (sexual preference data). alias is the only identity shown on platform.
companions	id, alias, city, vibe_tags[], fantasy_tags[], status, verified_at, created_at	Core companion row. fantasy_tags[] is the data backbone of the Bridge algorithm. GIN index on both tag arrays for fast overlap queries. status: active/pending/suspended.
companion_media	id, companion_id, type (photo/video/audio), storage_key, duration_sec, approved, order_index	All companion media. type determines display context. duration_sec enforced ≤15s for video at upload. approved = passed admin moderation.
companion_stories	id, companion_id, title, body_text, audio_url, fantasy_tags[], approved, published_at	Escort's own original content. fantasy_tags[] links story to Bridge algorithm. audio_url populated when escort uploads their own voice recording.
sessions_offered	id, companion_id, title, description, duration_mins, base_price_usd, session_type	Bookable session types per companion. session_type: in_person/audio_call/video_call/chat.
bookings	id, user_id, companion_id, session_id, status, stripe_payment_intent_id, scheduled_at, created_at	Full booking record. status: pending/confirmed/completed/cancelled/refunded. Stripe payment intent linked for reconciliation.
stories	id, title, body_text, audio_url, fantasy_tags[], orientation, author_type (user/escort/admin), author_id (nullable), approved, created_at	Platform-wide story feed. author_type determines display styling. audio_url set by BullMQ job after AI voicing.
ugc_posts	id, user_id (nullable=anonymous), type (text/audio), body_text, audio_url, fantasy_tags[], orientation, status, engagement_score, created_at	User confessions and stories. engagement_score computed daily (likes + saves + completions). High-scoring posts queued for AI voicing.
engagements	id, user_id, entity_type (story/companion/audio), entity_id, action (like/save/comment/share), created_at	Engagement events. Append-only. Partitioned by created_at. Feeds trending scores and collaborative filtering.
comments	id, entity_type, entity_id, body_text, approved, created_at	Fully anonymous comments. No user_id stored. Admin can delete. Nested replies max 1 level (reply_to_id).
user_behaviour	id, user_id, entity_type, entity_id, signal (scroll_depth/read_pct/listen_pct/hover/re_listen), value, session_id, created_at	Granular behavioural signals. Append-only. Feeds ML embedding updates. Partitioned by created_at for fast recent-window queries.
user_vectors	user_id, embedding vector(1536), updated_at	pgvector. User taste profile. Updated by BullMQ embedding job when behaviour signals accumulate. HNSW index for similarity search.
companion_vectors	companion_id, embedding vector(1536), updated_at	pgvector. Companion's profile vector (fantasy tags + bio + story content). Updated when companion adds content. HNSW index.

Table	Key Columns	Purpose & Design Notes
story_vectors	story_id, embedding vector(1536), updated_at	pgvector. Semantic vector for each published story. Used to find similar content and to match stories to companion profiles.
feed_cache	user_id, companion_ids jsonb, content_ids jsonb, computed_at	Materialised feed ranking per user. Stored in Redis (TTL 2hrs) and DB (for cold start fallback). Recomputed hourly by BullMQ cron job.
messages	id, room_id, sender_id, body_encrypted, seen_at, created_at	Phase 3. AES-256 encrypted message body. Room scoped per user-companion pair. Partitioned by created_at.
call_sessions	id, user_id, companion_id, type (audio/video), started_at, ended_at, billed_minutes, stripe_meter_event_id	Phase 3. Billing record. billed_minutes feeds Stripe Meter API. Linked to Stripe event for reconciliation.
push_subscriptions	id, user_id, endpoint, keys_json, created_at	Phase 3. Web Push API subscription objects per user device. Used by BullMQ notification job to send session reminders and message alerts.

6. Monorepo Project Structure

The entire codebase lives in a single GitHub repository managed with pnpm workspaces. This eliminates type drift between services — packages/shared ensures the frontend, API, and worker always speak the same schema language.

Path	Contents & Purpose
apps/web/	Next.js 14 App Router frontend. All pages, layouts, Server Components, Client Components. Deployed to Railway as blushbite-web service.
apps/api/	Hono.js + tRPC API server. All route handlers, middleware, auth callbacks. Deployed to Railway as blushbite-api service.
apps/worker/	BullMQ job workers. AI audio, embeddings, email, feed recalculation, moderation, push notifications. Deployed to Railway as blushbite-worker service.
apps/socket/	(Phase 3) Socket.io real-time chat server. Deployed to Railway as blushbite-socket service. Redis adapter configured.
packages/db/	Drizzle ORM schema (schema.ts) + migration files. Single source of truth for all database tables. Imported only by apps/api and apps/worker.
packages/shared/	Zod schemas, TypeScript types, utility functions, Hinglish phoneme map, alias wordlist, job type definitions. Imported by all apps. Zero runtime dependencies.
packages/ui/	Shared React UI components (feed card, companion card, audio player, save button, like button, comment thread) built on shadcn/ui. Imported only by apps/web.
packages/email/	React Email templates for all transactional emails. Imported only by apps/worker for email dispatch via Resend.
packages/hinglish/	Hinglish text pre-processor. Hindi word → IPA phoneme map. SSML injection logic for ElevenLabs. Imported by apps/worker.
.github/workflows/	GitHub Actions CI/CD. Pipeline: lint → type-check → test → build → deploy to Railway (separate deploy per service on main merge).
docker-compose.yml	Local development. Spins up Postgres (with pgvector), Redis, and mock Cloudflare R2 (via MinIO) with one command.
railway.json	Railway project configuration. Defines service build commands, health check paths, and environment variable references.

7. Phase-by-Phase Stack Activation

Not every service or feature is needed from Day 1. This section defines exactly what is live in each phase, so the team avoids unnecessary cost and complexity before it is required.

Phase 1 — What Is Live on Launch Day

Category	Services Active	Status
Railway Infrastructure	blushbite-web, blushbite-api, Postgres (pgvector), Redis, blushbite-worker	ACTIVE — all services deployed on Railway EU Frankfurt
Frontend	Next.js 14, Tailwind, shadcn/ui, Framer Motion, TanStack Query, Zustand	ACTIVE
Auth	NextAuth.js v5, Google OAuth, JWT httpOnly cookies, Age Gate middleware, Alias generator	ACTIVE
Companion Profiles	Photos (up to 20), 15s video clips (up to 5), audio stories, fantasy tags, vibe tags, session cards	ACTIVE — all formats supported from Day 1
Save / Like / Comment	Full engagement layer on all stories and escort content. Anonymous comments.	ACTIVE — feeds ML model from Day 1
Story-to-Escort Bridge	Static tag-overlap matching. 3 companion cards at bottom of every story page.	ACTIVE (static). Upgraded to ML-ranked in Phase 2
Cloudflare R2 + CDN	Media storage and global delivery for all photos, videos, audio	ACTIVE
Cloudflare Stream	Video transcoding and HLS delivery for 15s escort clips	ACTIVE
Monitoring	Sentry, BetterStack Uptime, Axiom logging	ACTIVE from Day 1
AI Voice (ElevenLabs)	N/A	NOT YET — Phase 2
ML Recommendations	N/A	NOT YET — Phase 2 (pgvector ready, data accumulates in Phase 1)
Live Payments (Stripe)	N/A	PARTIAL — Booking request form only. Full Stripe in Phase 2
Chat / WebRTC	N/A	NOT YET — Phase 3

Phase 2 — What Is Added

Addition	Technology	What It Enables
AI Voice (ElevenLabs)	ElevenLabs API + BullMQ audio job + R2 cache	Listen button on all stories. Hinglish support. Sticky audio player. Auto-voicing top UGC.
ML Recommendation Engine	OpenAI embeddings + pgvector HNSW + BullMQ cron	Feed ranked by user taste vector similarity. Bridge upgraded to ML-ranked. 'Because you liked X' rows.
Behavioural Analytics	PostHog + custom event pipeline	Scroll depth, listen %, read completion, hover events all captured. Feeds preference vector updates.
UGC Story & Audio Posting	Moderation queue + OpenAI Moderation API pre-screen	Users post anonymous confessions (text + audio). Auto-voicing of top posts within 24 hours.
Live Stripe Payments	Stripe full integration + BullMQ webhook handler	Booking commission, listing fees, companion payout system. All webhook events async via BullMQ.
Explore + Mix Builder	Full filter UI + recommendation endpoint	Full companion + content directory. Tonight's Mix Builder generates a personalised session feed on demand.
Admin Analytics Dashboard	PostHog funnels + Railway Postgres aggregations	Conversion funnels, Bridge CTR, audio completion rates, companion performance, trending content.

Phase 3 — What Is Added

Addition	Technology	What It Enables
Real-Time Chat	Socket.io on Railway + Redis adapter	1-to-1 private chat. Seen/typing indicators. First 5 messages free, then metered.
Presence System	Redis expiring keys + Pub/Sub	Companion online/available status. Auto-expires on disconnect.
Browser Audio Calls	WebRTC + Pion/mediasoup signalling + Coturn TURN	In-browser voice calls. Timer. Per-minute billing. No phone number sharing.
Browser Video Calls	WebRTC + TURN + Stripe Meter API	Full 1-to-1 video. Camera toggle. Pre-paid or metered billing. Highest ARPU feature.
Companion Scheduling	Availability calendar + slot booking UI	Companion sets available hours. User books specific slot. Confirmation + payment on-platform.
Push Notifications	Web Push API + Service Worker	Session reminders 10 min before start. New message alerts. No app download required.
Metered Billing	Stripe Meter API + call_sessions table	Per-minute charges for audio/video. Automatic charge at session end.

8. Developer Quick Reference

The most common development decisions and the definitive answer for each. Every team member should have this memorised.

Task / Question	Use This	Critical Notes
Build a page	Next.js Server Component	Default to RSC. Only add 'use client' if you need useState, useEffect, or browser events.
Style anything	Tailwind CSS className	No CSS files. No CSS-in-JS. All tokens in tailwind.config.ts.
Animate anything	Framer Motion motion.div	Use transform-only properties. Spring physics for interactions. AnimatePresence for unmount.
Fetch data on the server	Async RSC + Drizzle directly	No useEffect, no getServerSideProps. Parallel fetches with Promise.all.
Fetch data on the client	TanStack Query useQuery	Handles caching, loading, error, optimistic updates. Bootstrapped from server-prefetched data.
Global UI state	Zustand store	Auth, audio player, prefs, notification count. Not for server data.
Build an API endpoint	tRPC procedure in Hono	tRPC for type-safe frontend calls. Hono for Stripe webhooks and /health.
Query the database	Drizzle ORM	Never raw SQL in app code. Schema in packages/db/schema.ts.
Validate any input	Zod schema from packages/shared	Same schema on frontend and backend. Never duplicate validation logic.
Upload media (photo/video/audio)	Signed R2 URL flow	API generates signed URL. Client uploads direct to R2. API stores CDN URL in DB.
Generate AI audio	BullMQ audio job	Never call ElevenLabs inline. Enqueue job. Worker handles. Client polls or SSE.
Compute ML embeddings	BullMQ embedding job	Never call OpenAI inline. Enqueue on content create/update.
Send an email	BullMQ email job via Resend	Never send email in API handler. Enqueue async. Template in packages/email.
Heavy background task	BullMQ + apps/worker	Feed recalc, moderation pre-screen, push notifications all go here.
Handle auth in RSC	getSession() from NextAuth	Never decode JWT manually. getSession reads the secure cookie server-side.
Protect a route	Next.js middleware.ts	Runs before page renders. Checks JWT claims. Redirects unauthenticated users to /auth.
Real-time event (Phase 3)	Socket.io emit/on	Rooms per conversation. Redis adapter for multi-replica. JWT auth on connect.

Task / Question	Use This	Critical Notes
Track a user event	PostHog.capture('event')	Use anonymous alias hash as distinct_id. Never PII. Key events: story_listen, bridge_click, booking_start.
Store a secret	Railway Environment Variables	Never in code. Never in .env committed to Git. Shared vars across services via Railway shared variables.
Access another Railway service	Private network hostname	Format: \${ServiceName.ENV_VAR}. Zero latency internal routing.
Monitor an error	Sentry.captureException(err)	Wrap all API handlers in try/catch. Instrument critical flows as Sentry transactions.

The Guiding Rule for Every Technical Decision If it makes the user wait, make it async. If it can be cached, cache it at the edge. If it is heavy, put it in a BullMQ job. If it is a secret, put it in Railway env. If it is shared between frontend and backend, it lives in packages/shared.